

Artificial Intelligence Neural Networks

Basanta Joshi, PhD

Assistant Professor, Department of Electronics and Computer Engineering,
Pulchowk Campus <https://pcampus.edu.np/>

Deputy Director, Center for Applied Research & Development(CARD)
<https://card.ioe.edu.np/>

Member, Laboratory for ICT Research and Development (LICT) <http://lict.ioe.edu.np/>
Institute of Engineering, Tribhuvan University

basanta@ioe.edu.np

<http://www.basantajoshi.com.np>

<https://scholar.google.com/citations?user=iocLiGcAAAAJ>

https://www.researchgate.net/profile/Basanta_Joshi2



Slide | 1 By: Basanta Joshi, PhD

Introduction to Neural Nets

Introduction

to Neural Nets

Slide | 2  By: Basanta Joshi, PhD

The Human Brain

- Computers and the Brain: A Contrast

- Arithmetic: 1 brain = 1/10 pocket calculator – Vision: 1 brain = 1000 super computers – Memory of arbitrary details: computer wins
- Memory of real-world facts: brain wins

- A computer must be programmed explicitly
- The brain can learn by experiencing the world

Slide | 3 By: Basanta Joshi, PhD

Other Comparisons

ITEM COMPUTER BRAIN

Complexity ordered structure	Processor Speed 10,000,000
serial processor	operations per second

10 ¹⁰ neuron processors	operations per	
10 ⁴ connections	100	
Computational		second
Power	one operation at a time	millions of operations at a time
	1 or 2 inputs	thousands of inputs

Slide | 4 By: Basanta Joshi, PhD

Computer Operations

Traditionally computers execute a sequence of instructions to accomplish a set task

This is a powerful technique if you know the ‘algorithm’

It's not very useful if you don't !!

There are many interesting tasks where the algorithm is either unknown or unclear

- Recognizing handwriting - Pattern recognition
- Playing table tennis - Interacting with the environment
- Balancing activities - Optimization

- “. . . Neural nets are basically mathematical models of information processing . . .”
- “. . . (neural nets) refer to machines that have a structure that, at some level, reflects what is known of the structure of the brain . . .”
- “A neural network is a massively parallel distributed processor . . . “

Neural Net Concept

- Artificial Neural Systems are called:
 - neurocomputers
 - neural networks
 - parallel distributed processors
 - connectionists systems
- Basic Philosophy
 - large number of simple “neuron-like” processors which execute global or distributed computation

Slide | 8 By: Basanta Joshi, PhD

Processing Concept

INPUT
(Type)

OUTPUT

Processing

to data

(Procedural) Data Input

Procedures applied

1, 2, 3, etc.

Conventional

Data Output

(Logical) **Expert/Knowledge-Based System**
Facts Knowledge (Rules) Decision
(A is True) (Statistical) Recommend

Inference Engine & **Neural Networks**

Patterns Pattern Network Algorithm Identifies
Patterns recognized

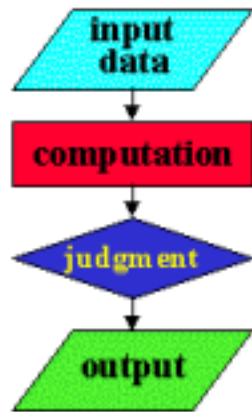
Slide | 9 By: Basanta Joshi, PhD

Features of Neural Nets

Neural Net Characteristics 1

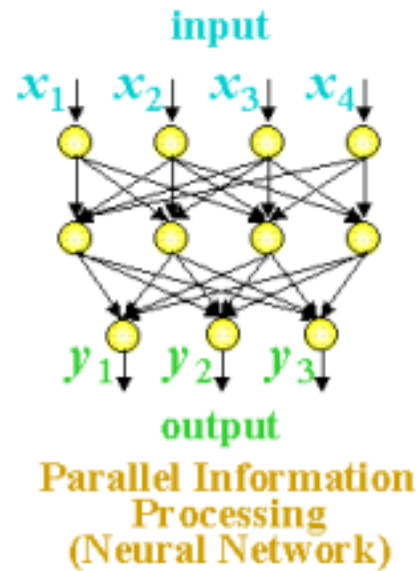
- Neural nets have many unique features:

Massive Parallel Processing:



Sequential Information
Processing (Computer)

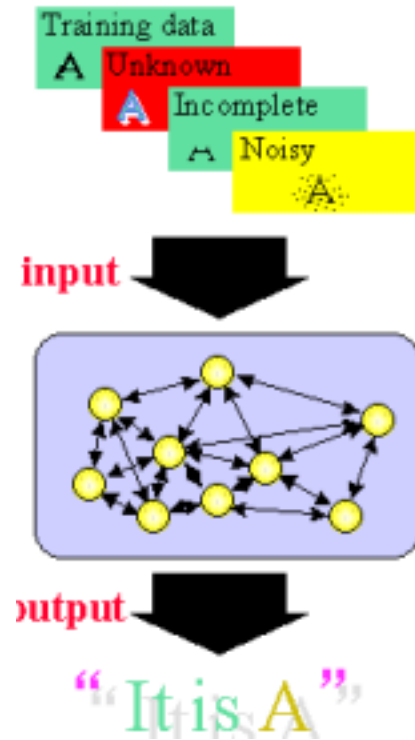
Neural Nets



update in parallelComputers calculate in sequence

Neural Net Characteristics 2

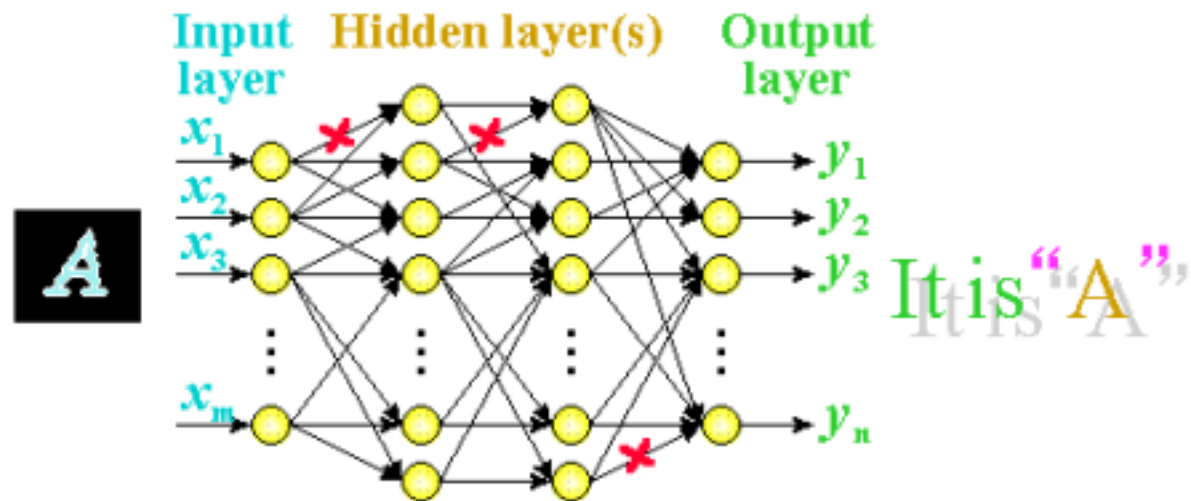
- Neural nets can learn:



Once a neural net is trained, it can handle unknown, incomplete or noisy data

Neural Net Characteristics 3

- Unlike computer memory, information is distributed over the entire network, so a failure in one part of the network does not prevent the system for producing the correct output.



Learning in Neural Nets

Learning in

Neural Nets

Slide | 13 By: Basanta Joshi, PhD

Learning

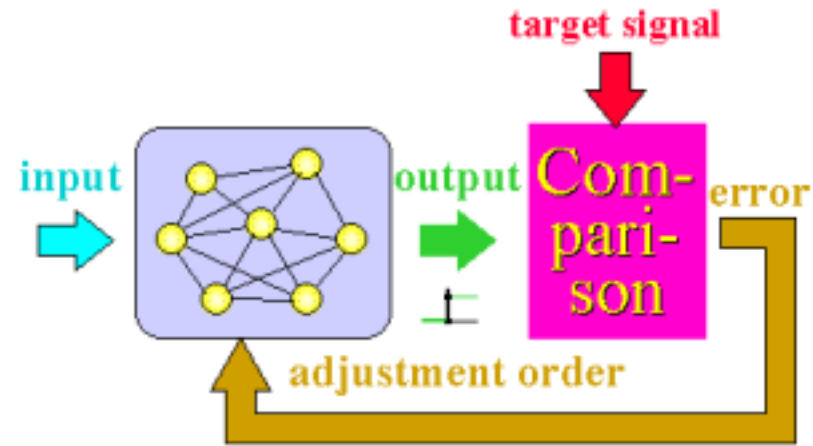
- One of the powerful features of neural networks is learning.

- Learning in neural networks is carried out by adjusting the connection weights among neurons.
- It is similar to a biological nervous system in which learning is carried out by changing synapses connection strengths, among cells.
- Learning may be classified into 2 categories:
 - (1) **Supervised Learning**
 - (2) **Unsupervised Learning**

- In supervised learning, the network is presented with inputs together with the target (teacher signal) outputs.
- Then, the neural network tries to produce an output as close as possible to the target signal by adjusting the values of internal weights.
- The most common supervised learning method is the “**error correction method**”

Error Correction Method

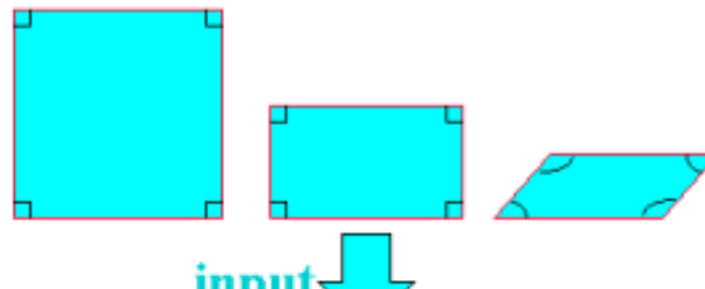
- Error correction method is used for networks which their neurons have discrete output functions.
- Neural networks are trained with this method in order to reduce the error (difference between the network's output and the desired output) to zero.



Slide | 16 By: Basanta Joshi, PhD

Unsupervised Learning

- In unsupervised learning, there is



no teacher (target signal) from outside and the network adjusts its weights in response to only the input patterns.

- A typical example of unsupervised learning is Hebbian learning.
- clustering
- hidden Markov models,
- self-organizing map (SOM) etc.

- In 1958, **Frank Rosenblatt** introduced a training algorithm that provided the first procedure for training a simple ANN: a **perceptron**.
- The perceptron is the simplest form of a neural network. It consists of a single neuron with *adjustable* synaptic weights and a *hard limiter*.
- The operation of Rosenblatt's perceptron is based on the **McCulloch and Pitts neuron model**. The model consists of a linear combiner followed by a hard limiter.
- The weighted sum of the inputs is applied to the hard limiter, which produces an output equal to +1 if its input is positive and -1 if it is negative.

Slide | 18 By: Basanta Joshi, PhD

Single-layer two-input perceptron *Inputs*

x_1

w_1

Linear
Combiner
Hard

Limiter

Output

$$\sum Y$$

Threshold

$$x_2$$

$$w_2$$

$$\theta$$

McCulloch/Pitts
Neuron

**McCulloch/Pitt
s Neuron**

Common Features

- “weight” = strength of connection
- threshold = value of weighted input below which no response is produced
- signals may be:
 - real-valued, or
 - binary-valued:

- “unipolar” $\{0, 1\}$
- “bipolar” $\{-1, 1\}$

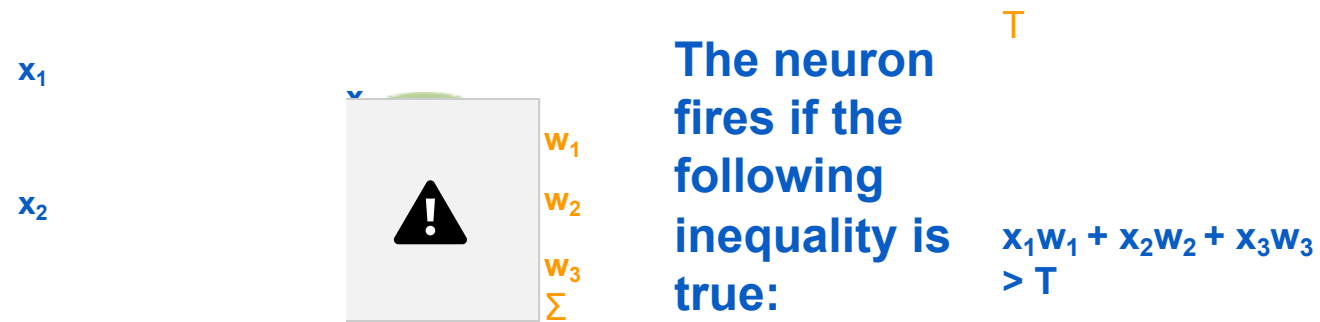
Slide | 21 By: Basanta Joshi, PhD

McCulloch/Pitts Neuron

- One of the first neuron models to be implemented
- Its output is 1 (fired) or 0
- Each input is weighted with weights in the range -1 to

+ 1

- It has a threshold value, T



Slide | 22 By: Basanta Joshi, PhD

Single Neuron Application

- **GOAL**: Construct an MCP (McCulloch/Pitts) neuron which will implement the function below:

PROBLEM: find the threshold, T , and the weights w_1 and w_2 where:

F is 1 if $x_1w_1 + x_2w_2 > T$

<u>x_1</u>	<u>x_2</u>	<u>F</u>	0	0	0	0	1	1	1	0	1	1	1
1													

Each line of the function table places conditions on the

What is this function?

unknown values: $0 < w_2 > T$

T x_1



w_1
 Σ

$w_1 > T$

$w_1 + w_2 > T$

w_1 and $w_2 = 0.7$ $T = 0.5$ works

w_2

x_2

- The aim of the perceptron is to classify inputs,
 - $x_1, x_2, \dots, x_n$, into one of two classes, say
 - A_1 and A_2 .
- In the case of an elementary perceptron, the n -dimensional space is divided by a hyperplane into two decision regions. The hyperplane is defined by the linearly separable function:

$$\sum_{i=1}^n w_i x_i - \theta = 0$$

$$i w_i$$

Slide | 24 By: Basanta Joshi, PhD

Perceptron

- How does the perceptron learn its classification tasks?
- This is done by making small adjustments in the weights to reduce the difference between the actual and desired outputs of the perceptron. The initial weights are randomly assigned, usually in the range $[-0.5, 0.5]$, and then updated to obtain the output consistent with the training examples.
 - If at iteration p , the actual output is $Y(p)$ and the desired output is $Y_d(p)$, then the error is given by:

$$e(p) = Y(p) - d^-$$

- Iteration p here refers to the p th training example presented to the perceptron.
- If the error, $e(p)$, is positive, we need to increase perceptron output $Y(p)$, but if it is negative, we need to decrease $Y(p)$.

Slide | 25 By: Basanta Joshi, PhD

The perceptron learning rule

$$w_i = w_i + \alpha \cdot e(p) \cdot x_i(p)$$

$$w_i(p+1) = w_i(p) + \alpha \cdot e(p) \cdot x_i(p)$$

where $p = 1, 2, 3, \dots$

α is the **learning rate**, a positive constant less than unity.

- The perceptron learning rule was first proposed by **Rosenblatt** in 1960. Using this rule we can derive the perceptron training algorithm for classification tasks.

Step 1: Initialisation

Set initial weights $w_1, w_2, \dots, w_n$ and threshold θ to random numbers in the range $[-0.5, 0.5]$.

If the error, $e(p)$, is positive, we need to increase perceptron output $Y(p)$, but if it is negative, we need to decrease $Y(p)$.

Perceptron's training algorithm

Step 2: Activation

Activate the perceptron by applying inputs $x_1(p)$, $x_2(p), \dots, x_n(p)$ and desired output $Y_d(p)$.

Calculate the actual output at iteration $p = 1$

$$= \sum_{i=1}^n -\theta \left[Y_p - \text{step} \left(\sum_{i=1}^n x_{ip} w_{ip} \right) \right]$$

where n is the number of the perceptron inputs,
and step is a step activation function.

Step 3: Weight training

Update the weights of the perceptron

$$w_i(p+1) = w_i(p) + \Delta w_i(p)$$

$$\Delta w_i(p) = \alpha \cdot x_i(p) \cdot e(p)$$

where $\Delta w_i(p)$ is the weight correction at iteration p .

The weight correction is computed by the **delta rule**:

$$\Delta w_i(p) = \alpha \cdot x_i(p) \cdot e(p)$$

$$w_i(p+1) = w_i(p) + \Delta w_i(p)$$

Step 4: Iteration

Increase iteration p by one, go back to *Step 2* and repeat the process until convergence.

Slide | 29 By: Basanta Joshi, PhD

Example of perceptron learning: the logical operation AND

Epoch-1	$x_1 x_2$	output- Y_d	weights	Actual	Error- e	weights
Inputs	Desired	Initial	$w_1 w_2$	output- Y	Final	$w_1 w_2$
2 3 4 5	0 1	0 1	0.3 0.2 -0.1	0.1	0 0 1 0	0.3 0.3 -0.1
0 0 1	0 1	0 0 0 1	0.2 0.2 0.0	0.1	0 0 0 1	-1 1 0.2 0.2 0.0
1	0 1	0 0 0 1	0.2 0.1 0.0	0.1	1	0 0.2 0.2 0.0
0 0 1	0 1	0 0 0 1	0.2 0.2 0.0	0.1	0 0 0 1	0 -1 0 0.1 0.2 0.0
1	0 1	0 0 0 1	0.2 0.1 0.0	0.1	0	0 0.2 0.2 0.0
0 0 1	0 1	0 0 0 1	0.1 0.1 0.0	0.1	0	0 0.1 0.1 0.0
1	0 1	0.3 0.3	0.1 0.1 0.0	0.1	0	0 0.1 0.1 0.0
0 0 1	0 1	0.3 0.2	-0.1 0.0	0 0 1 0	0	0.3 0.3 0.1 0.1 0.0
1	0 1	0.3 0.3	-0.1 0.0	0 0 0 1	-1 0	0.2 0.3 -0.1 0.0
0 0 1	0 1	0.3 0.3	-0.1 0.0	0	0	-0.1 0.1 1 0

0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1
~~Threshold: $\theta = 0.2$; learning rate: $\alpha = 0.1$~~

Slide | 30 By: Basanta Joshi, PhD

NN Development

development

- There is no single methodology for NN

Define Problem

– A suggested approach is: **Data Selection, Preparation, Preprocessing** **Determine Best-Suited**

Algorithm

Train Network

**Trained
Successfully?**

Test Network

**Tested
Successfully?
Run/Use Network**

Slide | 31 By: Basanta Joshi, PhD

Learning in Neural Nets (again)

- The operation of a neural network is determined by the values of the interconnection weights
 - There is no algorithm that determines how the weights should be assigned in order to solve a specific problems
 - Hence, the weights are determined by a learning process

Slide | 32 By: Basanta Joshi, PhD

Hebbian Learning

- The oldest and most famous of all learning rules is Hebb's postulate of learning:

“When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic changes take place in one or both cells such that A’s efficiency as one of the cells firing B is increased”

Application to artificial neurons:

If two interconnected neurons are both “on” at the same time, then the weight between them should be increased

Slide | 33 By: Basanta Joshi, PhD

Hebb’s Algorithm

- **Step 0:** initialize all weights to 0
- **Step 1:** Given a training input, s , with its target output, t , set the activations of the input units: $x_i = s_i$
- **Step 2:** Set the activation of the output unit to the target value: $y = t$
- **Step 3:** Adjust the weights: $w_i(\text{new}) = w_i(\text{old}) + x_i y$
- **Step 4:** Adjust the bias (just like the weights): $b(\text{new}) = b(\text{old}) + y$

Example

- **PROBLEM:**

Construct a Hebb Net which performs like an AND function, that is, only when both features are “active” will the data be in the target class.

- **TRAINING SET** (with the bias input always at 1):

x1	x2	bias	Target	1	1	1	1
1	-1	1	-1				
-1	1	1	-1	-1	-1	1	-1

1
b
 w^1_0

$x_1 \Sigma$
 w_2
 x_2

Slide | 35 By: Basanta Joshi, PhD

Training – First Input

- Initialize the weights to 0

Update the weights:

**Present the first input (1 1 1)
with a target of 1**

1

$$\begin{aligned}
 & \mathbf{b}(\text{new}) = \mathbf{b}(\text{old}) + \mathbf{t} = 0 + 1 = 1 \\
 & \mathbf{w}_1(\text{new}) = \mathbf{w}_1(\text{old}) + \mathbf{x}_1 \mathbf{t} = 0 + 1 = 1 \\
 & \mathbf{w}_2(\text{new}) = \mathbf{w}_2(\text{old}) + \mathbf{x}_2 \mathbf{t} = 0 + 1 = 1
 \end{aligned}$$

$\mathbf{x}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $\mathbf{x}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, $\mathbf{t} = 1$

Slide | 36 By: Basanta Joshi, PhD

Training – Second Input

- Present the second input:
 $(1 \ -1 \ 1)$ with a target of -
1

1

Update the weights:

$$w_1(\text{new}) = w_1(\text{old}) + x_1 t = 1 + 1(-1) = 0$$

$$w_2(\text{new}) = w_2(\text{old}) + x_2 t = 1 + (-1)(-1) = 2$$

$$b(\text{new}) = b(\text{old}) + t$$

$$1 + 1 = 2$$

$$x_1 \sum 1$$

$$1 - 1 = 1 + (-1) = 0$$

$$0 + 0 = 0$$

$$x_1 \sum x_2$$

Training – Third Input

Update the weights:

- Present the third input: $(-1 \ 1 \ 1)$ with a target of -1

$$w_1(\text{new}) = w_1(\text{old}) + x_1 t = 0 + (-1)(-1) = 1$$

$$w_2(\text{new}) = w_2(\text{old}) + x_2 t = 2 + 1(-1) = 1$$

$$b(\text{new}) = b(\text{old}) + t$$

1

-1

0
0

$$-1 = 0 + (-1) = -1$$

$x_1 \sum$

1

x_2

1

2

-1

1 0

$x_1 \sum$

1

x_2

Training – Fourth

Input • Present the fourth

input:
(-1 -1 1) with a target of -1

1

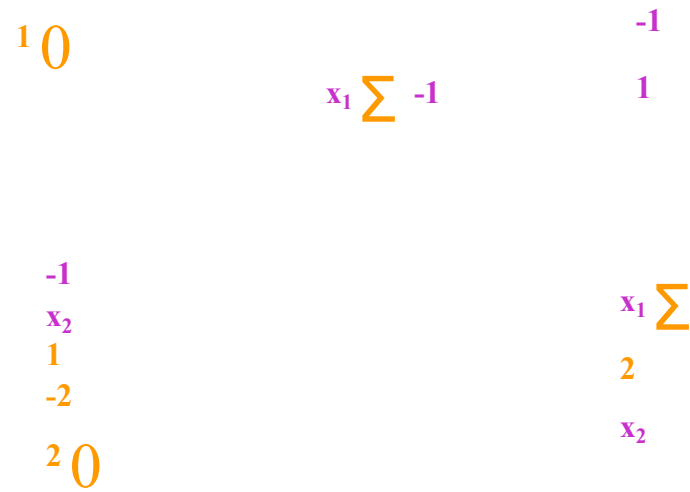
-1

Update the weights:

$$w_1(\text{new}) = w_1(\text{old}) + x_1 t = 1 + (-1)(-1) = 2$$

$$w_2(\text{new}) = w_2(\text{old}) + x_2 t = 1 + (-1)(-1) = 1$$

$$b(\text{new}) = b(\text{old}) + t \\ = -1 + (-1) = -2$$



Slide | 39 By: Basanta Joshi, PhD

Final Neuron

- This neuron works:

x1	x2	bias	Target
1	1	1	1

1 1 1 1 -2 1 1

1

1 1 -1 1 -1 -1

1 -1 1 $\underline{-1}$ $\underline{1}$ $\underline{-1}$ $\underline{1}$ -1 -1

-1 $^2 0$

-1 1 1 -1 -1 1 -1 1

1 -1

$x_1 \sum -1$

1

2

-1 -1 1 -1
-1 -1 1 -1

x_2

$$\begin{aligned}
 1*2 + 1*2 + 1*(-2) &= 2 > 0 & (-1)*2 \\
 + 1*2 + 1*(-2) &= -2 < 0 & 1*2 + \\
 (-1)*2 + 1*(-2) &= -2 < 0 & (-1)*2 + \\
 (-1)*2 + 1*(-2) &= -6 < 0
 \end{aligned}$$

Slide | 40 By: Basanta Joshi, PhD

OUTLINE

- Introduction to the Perceptron
- Adaline Network

The Perceptron

- The perceptron was suggested by Rosenblatt in 1958.

$$x_1 \sum$$

$$w_2$$

$$x_2$$

- It uses an iterative learning procedure which can be proven to converge to the correct weights for linearly separable data

Activation function: 0 if $-\Theta \leq s \leq \Theta$

$$\left\{ \begin{array}{l} 1 \text{ if } s > \Theta \\ -1 \text{ if } s < -\Theta \end{array} \right.$$

- It has a bias and a threshold function

$$b$$

$$w^1 \ominus$$

$$1 \text{ if } s > \Theta$$

$$f(s) =$$

Perceptron Learning Rule

- Weights are changed only when an error occurs
- The weights are updated using the following:

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i$$

t is either +1 or -1

α is the learning rate

If an error does not occur, the weights are

not changed

Slide | 42 By: Basanta Joshi, PhD

Limitations of the Perceptron

The perceptron can only learn to distinguish between classifications if the classes are *linearly separable*.

If the problem is not linearly separable then the behaviour of the algorithm is not guaranteed.

If the problem is linearly separable, there may be a number of possible solutions.

The algorithm as stated gives no indication of the quality of the solution found.

Slide | 43 By: Basanta Joshi, PhD

XOR Problem

- A perceptron network can not implement an XOR function

$w \ w \ w$

$+ \ + \ \leq$

$o(x_1, x_2)$

w_0
 $w_1 w_2 x_0 x_1 x_2$

0. 0. 0

0 1 2

$w w w$

+ + >

0. 1. 0

0 1 2

$w w w$

+ + >

1. 0. 0

0 1 2

INPUTS ARE DIFFERENT – 1

OUT $w w w$

+ + ≤

1. 1. 0

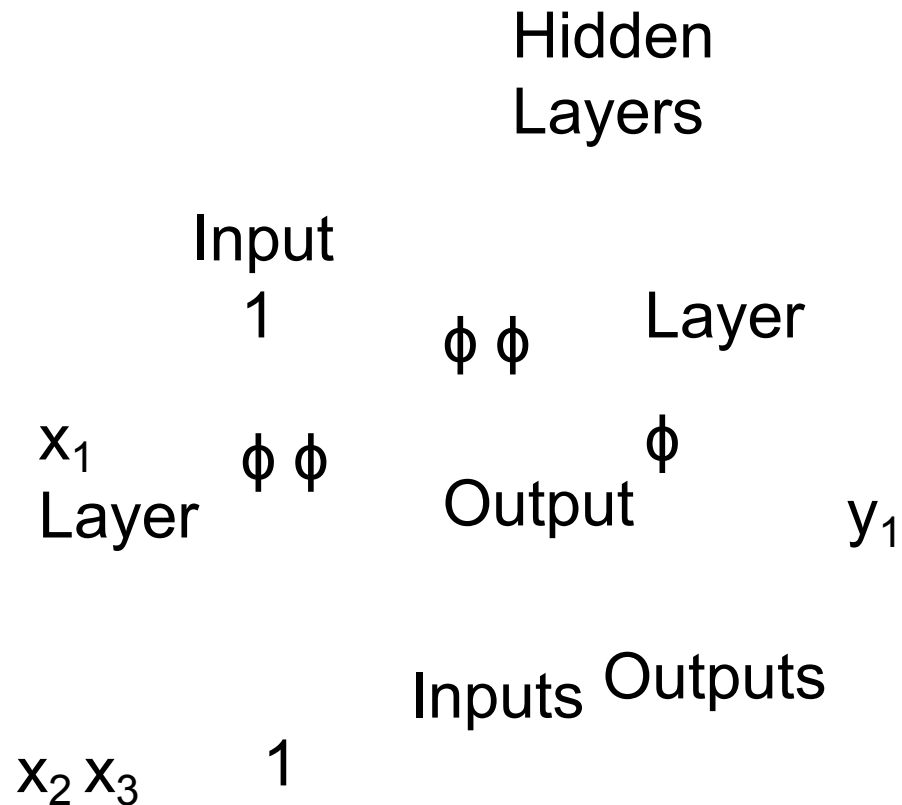
0 1 2

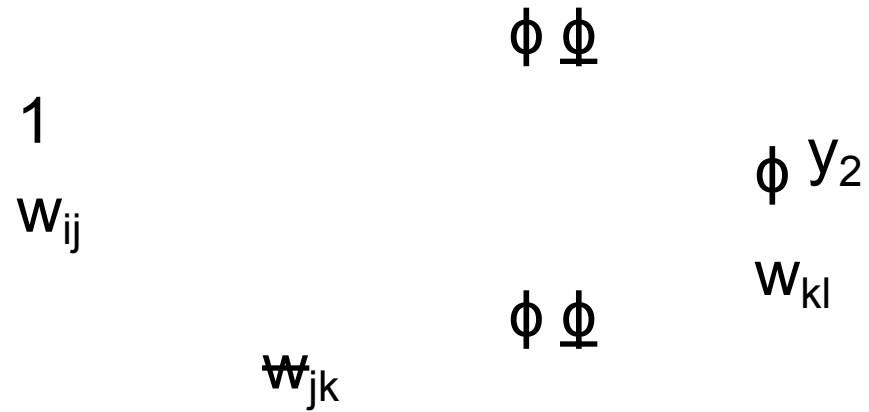
XOR(x_1, x_2)

There is no assignment of values to w_0, w_1 and w_2 that
~~satisfies above inequalities. XOR cannot be represented!~~

MultiLayer Perceptrons

- Perceptrons can be improved if placed in a multilayered network





Slide | 45 By: Basanta Joshi, PhD

Adaline Network

- Variation on the Perceptron Network
 - inputs are +1 or -1
 - outputs are +1 or -1
 - uses a bias input

- Differences

- trained using the Delta Rule which is also known as the least mean squares (LMS) or Widrow-Hoff rule
- the activation function, during training is the identity function
- after training the activation is a threshold function

Slide | 46 By: Basanta Joshi, PhD

Adaline Algorithm

- **Step 0: initialize the weights to small random values and select a learning rate, α**

- **Step 1:** for each input vector s , with target output, t set the inputs to s
- **Step 2:** compute the neuron inputs
- **Step 3:** use the delta rule to update the bias and weights
- **Step 4:** stop if the largest weight change across all the training samples is less than a specified tolerance, otherwise cycle through the training set again

Neuron input Delta rule

$$y_in = b + x \sum_i w_i$$

$$b(new) = b(old) + \alpha(t - y_in)$$

$$w_i(new) = w_i(old) + \alpha(t - y_in)x_i$$

Slide | 47 By: Basanta Joshi, PhD

The Learning Rate, α

- The performance of an ADALINE neuron depends heavily on the

choice of the learning rate

- if it is too large the system will not converge
 - if it is too small the convergence will take too long
- Typically, α is selected by trial and error
 - typical range: $0.01 < \alpha < 10.0$
 - often start at 0.1
 - sometimes it is suggested that:
$$0.1 < n\alpha < 1.0$$
where n is the number of inputs

Running Adaline

- One unique feature of ADALINE is that its activation function is different for training and running
- When running ADALINE use the following: – initialize the weights to those found during training – compute the net input – apply the activation function

Neuron input Activation Function

$$y_{in} = b + x \sum_i w_i y_i = \begin{cases} 1 & \text{if } y_{in} \geq 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

Slide | 49 By: Basanta Joshi, PhD

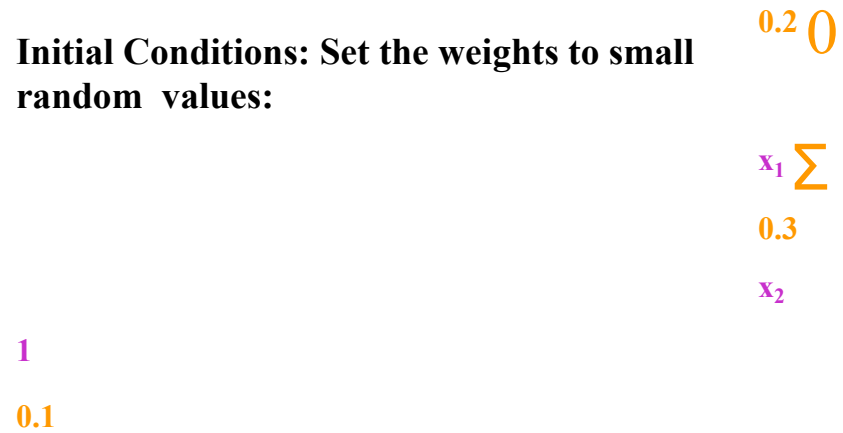
Example – AND function

- Construct an AND function for a ADALINE neuron –

```
let  $\alpha = 0.1$ 
```

[illegible]

Initial Conditions: Set the weights to small random values:



Slide | 50 By: Basanta Joshi, PhD

First Training Run

- Apply the input (1,1) with output 1

The net input is:

$$y_{in} = 0.1 + 0.2 \cdot 1 + 0.3 \cdot 1 = 0.6$$

$$b = 0.1 + 0.1(1 - 0.6) = 0.14$$

$$w_1 = 0.2 + 0.1(1 - 0.6) = 0.24$$

$$w_2 = 0.3 + 0.1(1 - 0.6) = 0.34$$

The new weights are: The largest weight change is 0.04

Neuron input Delta rule

$$y_{in} = b + \sum_i w_i x_i$$

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_{in})x_i$$

$$b(\text{new}) = b(\text{old}) + \alpha(t - y_{in})$$

Slide | 51 By: Basanta Joshi, PhD

Second Training Run

- Apply the second training set (1 -1) with output -1

The net input is:

$$y_{in} = 0.14 + 0.24*1 + 0.34*(-1) = 0.04$$

The new weights are: $w_1 = 0.24 - 0.1(1+0.04)1 = 0.14$

1

0.14

0.24 0

$b = 0.14 - 0.1(1+0.04) = 0.04$ The largest weight change is 0.1

0.44 -1

$$^1 \sum 0.34$$

-1

$$w_2 = 0.34 + 0.1(1+0.04)1 =$$

Third Training Run

- Apply the third training set (-1 1) with output -1

The net input is:

$$y_{in} = 0.04 - 0.14*1 + 0.44*1 = 0.34$$

The new weights are:

$$b = 0.04 - 0.1(1+0.34) = -0.09$$

1

0.04

0.14 0

-1 $\sum$ 0.44

1

$$w_1 = 0.14 + 0.1(1+0.34)1$$

$$= 0.27$$

-1

0.31

The largest weight change is 0.13

$$w_2 = 0.44 - 0.1(1+0.34)1 =$$

Slide | 53 By: Basanta Joshi, PhD

Fourth Training Run

- Apply the fourth training set (-1 -1) with output -1

1
-0.09
0.27 0

The net input is:

$$y_{in} = -0.09 - 0.27*1 - 0.31*1 = -0.67$$

$$-1 \sum 0.31$$

-1

The new weights are:

$$w_1 = 0.27 + 0.1(1+0.67)1 = 0.43$$

$$w_2 = 0.31 + 0.1(1+0.67)1 = 0.47$$

$$b = -0.09 - 0.1(1+0.67) = -0.27$$

The largest weight change is 0.16

Conclusion

- Continue to cycle through the four training inputs until the largest change in the weights over a complete cycle is less than some small number (say 0.01)
- In this case, the solution becomes: $b = -0.5$, $w_1 = 0.5$, $w_2 = 0.5$

Example – Recognize Digits

Slide | 55 By: Basanta Joshi, PhD

Multi-Layered Net (again)

neurons

output input

hidden layer

Properties

- Patterns of activation are presented at the inputs and the resulting activation of the outputs is computed.
- The values of the weights determine the function computed. A network with one hidden layer is sufficient to represent **every Boolean function**. With real weights **every real valued function** can be approximated with a single hidden layer.

Training

- Given Training Data,
 - input vector set :

$$\{ \mathbf{i}_n | 1 < n < N \}$$

- corresponding output (target) vector set:

$$\{ \mathbf{t}_n | 1 < n < N \}$$

- Find the weights of the interconnects using training data to *minimize*

the error in the test data

Slide | 58 By: Basanta Joshi, PhD

Error

- Input, target & response
 - input vector set : $\{ \mathbf{i}_n | 1 < n < N \}$
 - target vector set: $\{ \mathbf{t}_n | 1 < n < N \}$
 - $\mathbf{o}_n$ = neural network output when the

input is $\mathbf{i}_n$ • Error

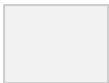


1

2



$o_n - t_n$



Error Minimization Techniques

- The error is a function of the
 - fixed training and test data
 - neural network weights
- Find weights that minimize error (Standard Optimization) –
 - conjugate gradient descent
 - random search
 - genetic algorithms
 - steepest descent (error backpropagation)



Backpropagation

- **Backpropagation is the most well know and widely used neural network system •**

It is a multi-layered, feedfoward, perceptron-like structure • Uses the

backpropagation rule (or generalized delta rule) for training

Slide | 61 By: Basanta Joshi, PhD
Slide | 62





Backpropagation

- Feedforward (for training and normal operation)
- Backpropagation (for error signals)



Slide | 63 By: Basanta Joshi, PhD

Backpropagation Algorithm Training 1

- **Step 0: Initialize the weights to small random values**

- **Step 1: Feed the training sample through the network and determine the final output** •

Step 2: Compute the error for each output unit, for unit k it is:

$$\delta_k = (t_k - y_k) f'(y_{in_k})$$

Actual output
Required output

Derivative of f

- Step 3: Propagate the delta terms (errors) back through the weights of the hidden units where the delta input for the j^{th} hidden unit is:

$$\delta_{\text{in}_j} = \sum_{k=1}^m \delta_k w_{jk}$$

The delta term for the j^{th} hidden unit is:

$$\delta_j = \delta_{\text{in}_j} f'(z_{\text{in}_j})$$

Backpropagation Algorithm Training 3

- Step 4: Calculate the weight correction term for each output unit, for unit k it is:

Hidden layer signal

$$\Delta w_{jk} = \alpha \delta_k z_j$$

A small constant

- Step 5: Calculate the weight correction term for the hidden units:

$$\Delta w_{ij} = \alpha \delta_j x_i$$

Slide | 66 By: Basanta Joshi, PhD

Backpropagation Algorithm Training 4

- Step 6: Update the weights:
-

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \Delta w_{ij}$$

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \Delta w_{jk}$$

- Step 7: Test for stopping (maximum cycles, small changes, etc)

Slide | 67 By: Basanta Joshi, PhD

Options

- There are a number of options in the design of a backprop system
 - Initial weights – best to set the initial weights (and all other free parameters) to random

- numbers inside a small range of values (say -0.5 to 0.5)
- Number of cycles – tend to be quite large for backprop systems
- Number of neurons in the hidden layer – as few as possible

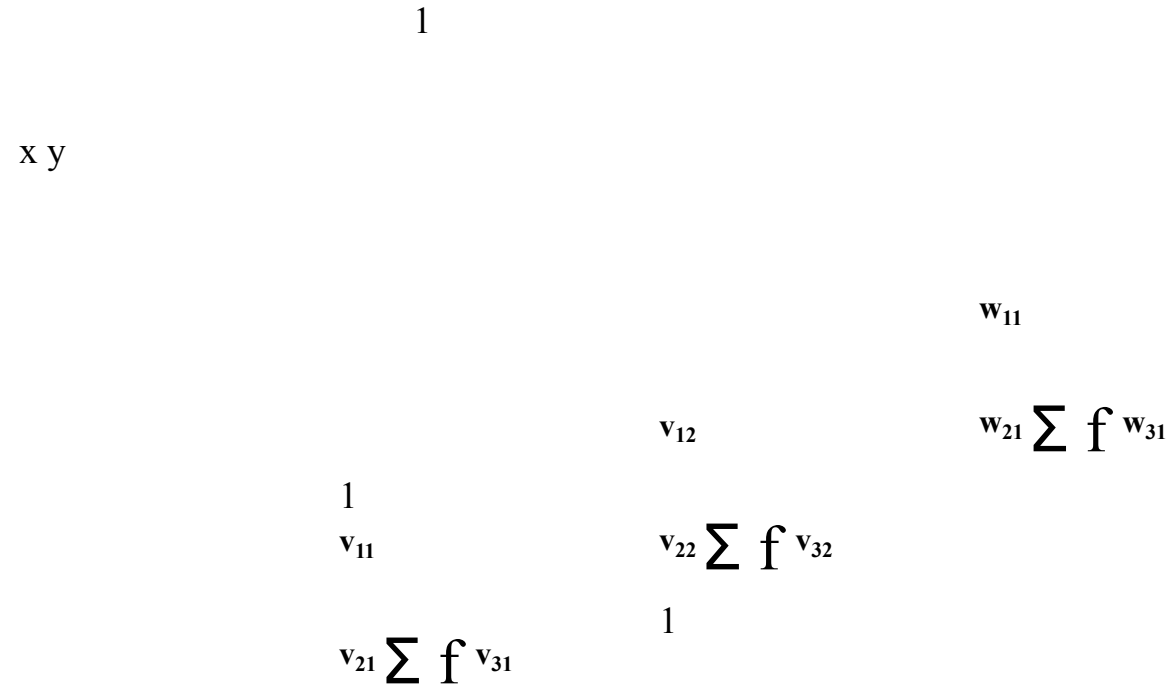
Example

- The XOR function could not be solved by a single layer perceptron network •

The function is:

X	Y	F
0	0	0
0	1	1
1	0	1
1	1	0

XOR Architecture



Slide | 70 By: Basanta Joshi, PhD

Initial Weights

- Randomly assign small weight values:

1

-.3

1

x y

$$-.4$$
$$-.2 \sum f_{.3}$$

$$1$$
$$.21 \sum f_{.15}$$
$$.25$$
$$-.4 \sum f_{.1}$$

Slide | 71 By: Basanta Joshi, PhD

Feedforward – 1st Pass

1

$$1 \quad \quad \quad \mathbf{-.318}$$

$$\mathbf{-.4}$$

x

$$\mathbf{.1}$$

0

1

1

$$\mathbf{s_1 = -.3(1) + .21(0) + .25(0) =}$$
$$\mathbf{-.3 \quad f = .43}$$

y

Training Case: (0

0

0 0)

$$\mathbf{f = .56}$$

$$\mathbf{-.2 \sum f}$$

Activation

1

$$\mathbf{.3}$$

function f: 1

1

$$\mathbf{-.3}$$

$$\mathbf{s_2 = .25(1) -.4(0) + .1(0)}$$

$$\mathbf{.21 \sum f}$$

$$\mathbf{.25}$$

$$\mathbf{y_j = s_j}$$

$$\mathbf{.15}$$

$$\mathbf{-.4 \sum f}$$

$$\mathbf{1 + e}$$

$$\mathbf{f = .42}$$

$$\mathbf{s_3 = -.4(1) - .2(.43) + .3(.56) =}$$
$$\mathbf{(not \ 0)}$$

Backpropagate

$$\delta_3 = (t_3 - y_3)f'(y_{in_3})$$
$$= (t_3 - y_3)f(y_{in_3})[1 - f(y_{in_3})]$$
$$\delta_{in_1} = \delta_3 w_{13} = -.102(-.2) = .02$$
$$\delta_1 = \delta_{in_1} f'(z_{in_1}) = .02(.43)(1-.43) = .005$$
$$\delta_{in_2} = \delta_3 w_{12} = -.102(.3) = -.03$$
$$\delta_3 = (0 -$$

$$1$$
$$0$$
$$-.3$$
$$.21 \sum f$$
$$.15$$
$$1$$
$$-.4$$
$$-.2 \sum f$$
$$.3$$
$$.1$$
$$-.4 \sum f$$

$$.42).42[1-.42] = \quad -.102$$

$$\delta_2 = \delta_{in_2} f'(z_{in_2}) = -.03(.56)(1-.56) = -.007$$

Slide | 73 By: Basanta Joshi, PhD

Update the Weights – First

Pass $\delta_3 = (t_3 -$

$y_3)f'(y_{in_3})$

1

$.21 \sum f .15$

0

-.3

$$=(t_3 - y_3)f(y_{in_3})[1 -$$

$$f(y_{in_3})]$$

1

0

$$\delta_{in_1} = \delta_3 w_{13} = -.102(-.2) = .02$$

$$-.4 \sum f$$

.1

$$\delta_{in_2} = \delta_3 w_{12} = -.102(.3) = -.03$$

$$\delta_2 = \delta_{in_2} f'(z_{in_2}) = -.03(.56)(1-.56)$$

Slide | 74 By: Basanta Joshi, PhD = -.007

$$\delta_1 = \delta_{in_1} f'(z_{in_1}) = .02(.43)(1-.43) = .005$$

$$.25$$

$$-.2 \sum f$$

$$\delta_3 = (0 - .42).42[1-.42] = -.102$$

Final Result

- After about 500 iterations:

1

1

-1.5

x y

$\sum_{i=1}^n f_i$

$$\begin{matrix}
 -.5 \\
 {}^1 \sum f_1 \\
 -.5 \\
 {}^{-2} \sum f_1
 \end{matrix}$$

Slide | 75 By: Basanta Joshi, PhD

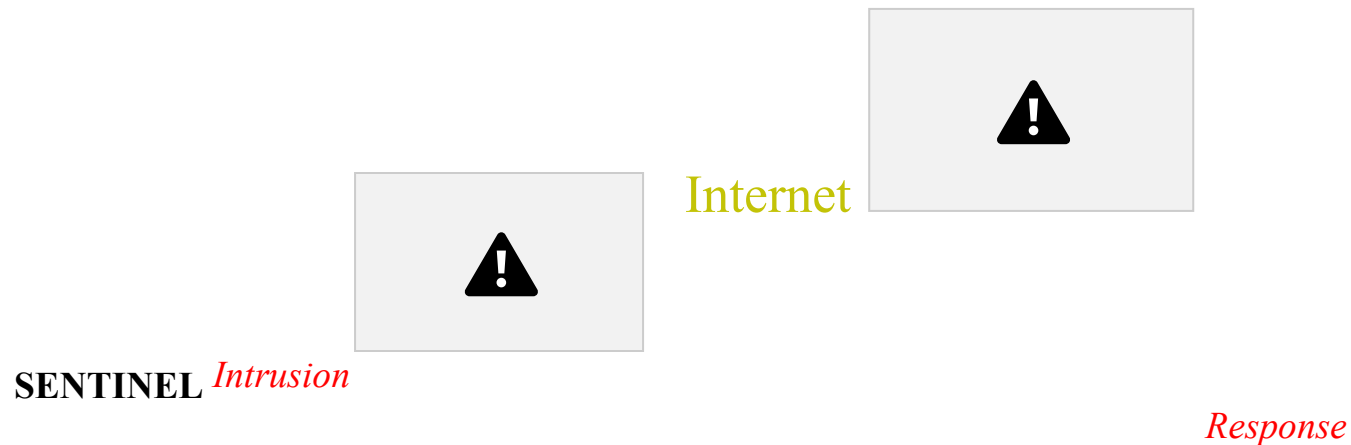
ppcaons Intrusion Detection ~~Systems~~

- **Anomaly Detection** - The identification of unauthorized system usage

by discovering statistical variances from established norm, (e.g., profiles of authorized users). Primarily used for detecting internal intrusions.

- **Misuse Detection** - The detection of external attacks (“hackers”, etc.). Traditionally addressed by matching current activities to established attacks patterns.

- GOAL: Initial design and development of a neural network-based misuse detection system which explores the use of this technology in the identification of instances of external attacks on a computer network.



Slide | 77 By: Basanta Joshi, PhD

Thank you !!!

Slide | 78 By: Basanta Joshi, PhD